

ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.



SWE 4601 - SOFTWARE DESIGN AND ARCHITECTURES

Quiz 4

Written by - Namisa Najah Raisa
Student ID: 210042112

Sixth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

September 24, 2025

Abstract

This document demonstrates the application of SWE 4601 learnings to my DP I project — **CodERA**. It details concrete design improvements including service-based architecture. These examples illustrate how theoretical design principles and architectural strategies can be directly mapped to a working platform to improve modularity, scalability, maintainability, and user experience.

In addition, this document also reimagines the course **SWE 4601 — Software Design & Architecture** from the perspective of a learner-turned-instructor. Although not a professional educator, I have designed this course in the way I would have most enjoyed learning it: through active labs, collaborative problem-solving, and reflection that ties abstract theory to real-world scenarios.

Contents

1	Leveraging SWE-4601 Learnings for Project Refinement	4
1.1	Repository snapshot	4
1.2	Elevator pitch	4
1.3	Concrete improvements	4
1.3.1	Software architecture: component / service-based design	4
1.3.2	Connectors / Middleware / Message Queue	4
1.3.3	Design Philosophies & Principles	4
1.3.4	UI/UX	5
1.3.5	UML and Design Artifacts	6
1.3.6	Non-Functional Requirements (NFRs) & Trade-offs	6
1.3.7	Design Evaluation & Usability Testing	6
1.3.8	Software Redesign & Architectural Migration	6
1.4	Prioritized action list	6
1.5	Sample pseudocode: message flow	6
2	Instructional Design Perspective for SWE 4601	7
2.1	Course Philosophy	7
2.2	Learning Outcomes	7
2.3	Teaching and Learning Approach	7
2.3.1	Weekly Labs	8
2.3.2	Group Home Tasks	8
2.3.3	Workshops and Debates	8
2.3.4	Interactive Challenges	8
2.3.5	Reflection Reports	8
2.4	Assessment Design	8
2.5	Support Structures	9
2.6	Closing Perspective	9

1 Leveraging SWE-4601 Learnings for Project Refinement

1.1 Repository snapshot

CodERA is an ambitious learning and coding platform featuring authentication, a compiler, an AI chatbot, progress/heatmaps, and real-time collaboration. This plan focuses on increasing modularity, scalability, security, and user experience while aligning with SWE-4601 topics such as software architecture, design patterns, UI design, and architectural migration. **Example: Currently, all compilation requests hit the main API, which risks blocking when multiple users submit jobs simultaneously. The improvement extracts the compile workload into a dedicated service.**

1.2 Elevator pitch

Re-architect the app for *modularity, robustness, and UX polish*. Extract long-running responsibilities into isolated services, adopt proven design patterns, add UML artifacts, and apply the Golden Rules of UI design to improve user retention and developer velocity. **Example: Implement a message queue-based compile pipeline to decouple front-end submission from backend compilation, improving responsiveness for multiple concurrent users.**

1.3 Concrete improvements

1.3.1 Software architecture: component / service-based design

- **Action:** Extract the Compiler and Code Analyzer into a separate `compile-service` container.
- **Why:** Isolates long-running tasks, prevents request blocking, enables independent scaling.
- **Example flow:**
 1. Frontend POSTs to `/api/submissions`.
 2. API publishes a `submission.created` message to a message queue.
 3. `compile-service` consumes the message, runs a sandboxed compilation, and publishes `submission.finished`.
 4. Web API persists the result and notifies the user via WebSocket.
 5. **Concrete improvement:** Multiple users can now submit jobs simultaneously without blocking other requests.

1.3.2 Connectors / Middleware / Message Queue

- Add an API Gateway for routing and rate limiting. **Example: prevents overload from repeated requests during peak usage.**
- Centralize JWT-based auth into an Auth middleware used by all services. **Example: removes repeated validation code and avoids accidental unauthenticated routes.**
- Standardize inter-service message schemas (JSON Schema or Protocol Buffers) and include versioning metadata. **Example: all workers understand message format consistently, reducing runtime errors.**

1.3.3 Design Philosophies & Principles

- **SOLID**

- Single Responsibility: keep compiler orchestration separate from request routing. **Example: separates CompilerStrategy logic from API routing, making each module easier to test.**
- Open/Closed: make compiler strategies extensible without modifying existing code. **Example: adding a Rust compiler only requires implementing RustCompiler class.**
- Liskov Substitution: compiler interface implementations are interchangeable. **Example: CppCompiler or PythonCompiler can be swapped without changing orchestration.**
- Interface Segregation: small, focused interfaces. **Example: ICompiler only has compile() and getLogs() methods, no unrelated functions.**
- Dependency Inversion: depend on abstractions, not concrete classes. **Example: API controllers depend on ICompiler interface, not specific language classes.**
- **YAGNI (You Aren't Gonna Need It)**
 - Implement a single language pipeline first; add others only when required. **Example: Remove untested Java stub until officially needed, reducing unnecessary maintenance.**
- **KISS (Keep It Simple, Stupid)**
 - Use simple, well-understood solutions. **Example: leverage RabbitMQ dead-letter queue instead of writing a custom retry scheduler.**
- **POLA (Principle of Least Astonishment)**
 - APIs and UI behavior should match expectations. **Example: return structured compile errors like {error: "CompilationError", line: 4, message: "missing semicolon"} instead of vague messages.**
- **DRY (Don't Repeat Yourself)**
 - Centralize validation and error-handling. **Example: validation logic for submissions is now in validateSubmission() utility instead of repeated in every route.**
- **Minimal Coupling**
 - Components communicate via well-defined interfaces. **Example: API only interacts with compile-service via versioned message contracts; no internal state leaks.**

1.3.4 UI/UX

- Apply heuristics: visibility of system status, error prevention, and consistent mappings. **Example: progress bar now updates live via WebSocket for every compilation step.**
- Replace blind spinners with streaming compilation logs. **Example: users see "Compiling main.cpp... Error at line 5" instantly instead of waiting 10s.**
- Improve heatmap dashboard with legends and tooltips. **Example: color intensity now clearly indicates user activity; tooltips show submission counts.**

1.3.5 UML and Design Artifacts

- Class diagram for domain objects: User, Course, Submission, CompilerJob. **Example: diagrams clarify relationships for new contributors.**
- Sequence diagrams for registration and submission-to-compile flows. **Example: clearly documents messaging between frontend, API, queue, and compile-service.**
- Component and deployment diagrams showing services, queues, databases, and CDN. **Example: identifies scaling points and deployment dependencies.**
- Include PlantUML source files for version control.

1.3.6 Non-Functional Requirements (NFRs) & Trade-offs

- Measurable NFRs: Availability 99.5%, Compile latency 95% ; 8s. **Example: benchmark existing pipeline and optimize caching.**
- Trade-offs: Event-driven architecture improves scalability but adds operational complexity. **Example: logging and monitoring pipelines must handle distributed service tracing.**

1.3.7 Design Evaluation & Usability Testing

- Instrument analytics and run small usability tests. **Example: 5 users completed onboarding while monitoring click paths to optimize navigation.**
- A/B test progress bar wording to increase course completion. **Example: "Processing" vs "Compiling" label tested to see which reduces confusion.**

1.3.8 Software Redesign & Architectural Migration

- Use Strangler Fig pattern to gradually move features from monolith to microservices. **Example: initially route only /submissions to compile-service.**
- Implement canary deploys and feature flags. **Example: enable new dashboard only for 10% of users to verify stability.**

1.4 Prioritized action list

1. Extract the compile pipeline into compile-service with RabbitMQ / Redis Streams.
2. Add streaming logs to front-end via WebSocket.
3. Create three UML diagrams (class, sequence, component) and add PlantUML sources.
4. Refactor backend into Repository + Service layers and implement CompilerStrategy implementations.
5. Run a UI heuristics audit and iterate on the top 3 issues.
6. Add a documented NFR section in README with SLAs and security checklist.
7. Instrument analytics for two-week data collection and analysis.

1.5 Sample pseudocode: message flow

```
1 // web-api
2 POST /submissions
```

```

3   validate -> save Submission(status=PENDING) -> publish {type: "
      submission.created", id, code}
4
5   // compile-service (consumer)
6   on "submission.created":
7     pull job -> run sandboxed docker -> publish {type: "submission.
      finished", id, result, logs}
8
9   // web-api (consumer)
10  on "submission.finished":
11    persist result -> notify via websocket channel "submission.{id}
      "

```

2 Instructional Design Perspective for SWE 4601

2.1 Course Philosophy

If I were the instructor of this course, I would approach it as a journey of practice, discussion, and discovery. Students often encounter architecture as a dry subject, full of diagrams and abstract terminology. To counter this, my version of the course emphasizes active engagement: every topic must lead to a concrete artifact, a lively debate, or a reflective insight.

1. **Learn by doing:** Theories only stick when applied immediately. Every lecture would be paired with a lab or workshop.
2. **Design is storytelling:** Students should not only design but also learn to justify trade-offs, explain design rationale, and connect architecture to user needs.
3. **Collaboration mirrors reality:** Real systems are not built alone. Students would regularly work in teams, practicing communication, negotiation, and shared responsibility.

2.2 Learning Outcomes

By the end of the course, students should be able to:

1. Translate informal requirements into structured architectural solutions.
2. Model both structure and behavior of systems using industry-standard notations.
3. Compare alternative architectures and defend chosen trade-offs.
4. Apply patterns and styles with purpose, avoiding superficial use.
5. Critique peer designs and provide meaningful improvements.
6. Reflect on how non-functional requirements shape real-world architecture.

2.3 Teaching and Learning Approach

The teaching approach follows a repeating cycle:

Concept introduction → Guided lab → Group home task → Reflection and critique.

This ensures that every piece of theory is transformed into practice, reinforced by teamwork, and consolidated through structured reflection.

2.3.1 Weekly Labs

Weekly labs serve as the backbone of learning. For example:

- After introducing UML sequence diagrams, the lab would require students to model a login flow of a real application.
- When studying design patterns, students would reimplement a simple class in two different ways (with and without the pattern) and compare the trade-offs.

2.3.2 Group Home Tasks

Following each lab, students would complete a small group-based assignment. These tasks would encourage extension and application, such as:

- Expanding an in-class UML diagram into a full model of a subsystem.
- Designing a clickable low-fidelity prototype to complement diagrams.
- Writing a short reflection explaining the impact of one chosen non-functional requirement.

2.3.3 Workshops and Debates

Certain weeks would feature design studio workshops where teams critique each other's solutions, or debate circles where two teams argue for and against an architectural style. The aim is to make architecture less about "memorizing definitions" and more about "reasoning and communication."

2.3.4 Interactive Challenges

Light gamification would be included to reinforce learning, such as:

- Where students identify patterns in everyday software (e.g., Netflix, GitHub).
- Quick-fire architecture puzzles where students must choose an appropriate style for a fictional system in five minutes.

2.3.5 Reflection Reports

Each week, students would write a short reflection report connecting what they practiced to real-world applications. This reinforces self-awareness and ensures concepts are not forgotten after the lab.

2.4 Assessment Design

The assessment scheme emphasizes practical work and collaboration.

- **Labs (20%):** Weekly submissions of diagrams, prototypes, or configurations.
- **Group Home Tasks (15%):** Collaborative assignments extending lab work into larger deliverables.
- **Design Sprints (20%):** Short, timed in-class challenges culminating in peer critique.
- **Capstone Project (35%):** A semester-long project where teams design, prototype, and present a complete system.
- **Engagement and Peer Feedback (10%):** Assessed through participation in critiques, debates, and workshops.

This distribution ensures balance between individual accountability, teamwork, and long-term project growth.

2.5 Support Structures

To make the course more approachable, several support mechanisms would be embedded:

- GitHub Classroom to streamline submissions and encourage version control habits.
- Lab handbooks to guide students step-by-step while still leaving room for creativity.
- Weekly design clinics (office hours) where students can troubleshoot with the instructor or TA.
- Peer review forms to structure feedback and prevent vague or unhelpful critiques.

2.6 Closing Perspective

If I were teaching this course, I would want it to feel less like a rigid sequence of lectures and more like a studio environment where ideas are built, tested, and refined collaboratively. Architecture, at its heart, is about reasoning and trade-offs. Through labs, teamwork, debates, and reflection, students would not only learn the technical skills but also the communication and judgment that define a true software architect.

Most importantly, I would hope students leave the course not only knowing *what* architectures look like, but also understanding *why* design decisions matter, and how to explain those decisions clearly and confidently.